MECHATRONIC SYSTEM INTEGRATION
(MCTA 3203)

SEMESTER 1, 2025/2026

EXPERIMENT 5 :

Smart Surveillance System with Motorized Camera Base (ver 1.0)

SECTION 2

GROUP 12

NAME	METRIC NO.
AHMAD FARHAN BIN AHMAD NAHRUDI	2317373
MUHAMMAD AIMAN BIN SAFAWI	2318249
MOHD NAFIZ ZUHAIREE BIN MOHD NEEZAR	2317423

DATE OF SUBMISSION :
19/11/2025

ABSTRACT

This experiment centered on building a smart surveillance system using the ESP32-CAM by completing three key tasks: adding a pushbutton for manual panning control, activating the module's built-in face detection, and introducing vertical movement through a second servo motor. The system combined Wi-Fi video streaming, sensor processing, and PWM-based servo control to form an IoT-ready camera platform capable of moving in both horizontal and vertical directions. Live video was transmitted over Wi-Fi, while additional control logic enabled the system to react to user inputs or visual cues. Through these tasks, the experiment showed how a microcontroller, servo motors, and basic vision-processing features can work together to form a functional surveillance prototype. The results also demonstrate the ESP32-CAM's potential for low-cost monitoring solutions and highlight essential mechatronic principles, including real-time processing, hardware–software interfacing and integrated embedded system design.

TABLES OF CONTENTS

TABLES OF CONTENTS
ABSTRACT.....2
TABLES OF CONTENTS.....3
1.0 INTRODUCTION.....5
2.0 MATERIAL AND EQUIPMENT.....6
3.0 EXPERIMENTAL SETUP.....7
3.1 Setup Preparation..... 7
3.2 Pin Connections (Typical for Motor A)..... 7
3.3 Programming.....7
3.4 Initial Testing.....7
3.5 PWM Speed Control Test.....7
3.6 Data Recording & Analysis.....8
3.7 Shutdown.....8
4.0 METHODOLOGY.....9
4.1 Objectives……9
4.2 Implementation and Testing..... 10
4.3 Control Algorithm..... 11
4.4 Code Used..... 13

5.0 DISCUSSIONS..... 16
5.1 Task..... 16
6.0 CONCLUSION.....17
7.0 RECOMMENDATION.....18
REFERENCES..... 19
APPENDICES..... 19
ACKNOWLEDGEMENT..... 20
STUDENTS DECLARATION..... 20
Certificate of Originality and Authenticity.....20

1.0 INTRODUCTION

Smart surveillance systems are becoming an important part of modern mechatronics because they combine embedded computing, wireless communication, and automated control in one platform. In this experiment, the ESP32-CAM was used as the main device. It is a small microcontroller with an OV2640 camera and built-in Wi-Fi, allowing real-time video streaming, remote viewing, and basic onboard processing. This makes it a suitable tool for learning the basics of IoT-based vision systems.

The lab focused on expanding the ESP32-CAM's functions by adding actuation and simple user-interaction features. Servo motors were used to create horizontal and vertical camera movement through PWM control. A pushbutton was included to manually trigger motion, showing how digital inputs and basic control logic work. The experiment also explored the module's face detection feature, demonstrating how embedded systems handle light image-processing tasks. Through Tasks 1, 2 and 3, this practical session gave hands-on experience in applying mechatronic concepts to a realistic surveillance setup, highlighting the importance of system integration and responsive design in modern embedded applications.

2.0 MATERIAL AND EQUIPMENT

The following materials and equipment were used during this laboratory session, consistent with the experiment documentation provided:

- Microcontroller Board (Arduino Mega)
- Pixy Camera Module (For real-time color/object detection)
- Servo Motors (two for Pan and Tilt)
- 5V DC Power Supply
- Pushbutton 3x
- Mounting Bracket / Servo Base (To physically enable 2D camera movement)
- Breadboard
- Jumper Wires (Several)

3.0 EXPERIMENTAL SETUP

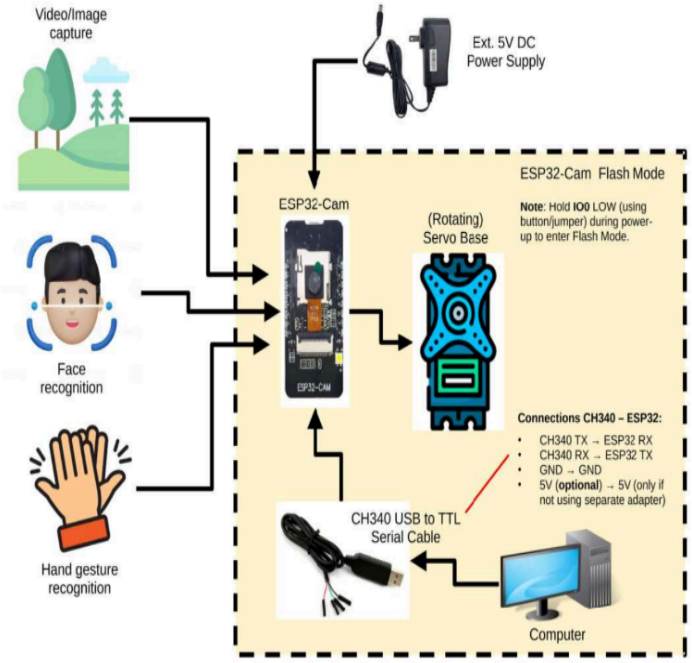

Figure 3.1 System Overview

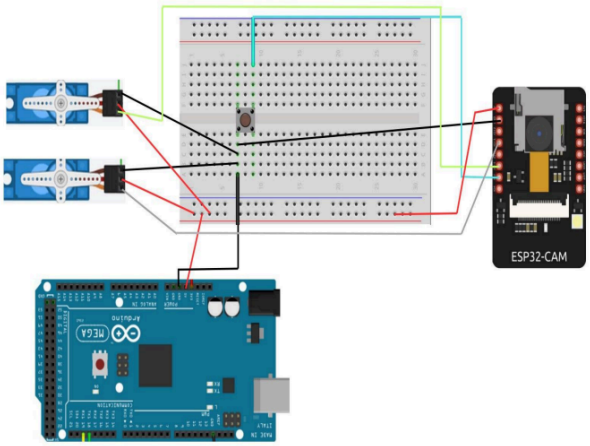

Figure 3.2 Schematic Diagram Smart Surveillance System

The circuit was assembled to connect the ESP32-CAM module with two servo motors and a pushbutton, allowing manual panning and vertical camera movement. An ESP-to-Serial programmer was also included to upload the firmware and establish initial communication with the microcontroller.

For Task 1, the circuit connections were arranged as follows:

•The servo motor's signal wire was connected to GPIO 12 on the ESP32-CAM for PWM control.

•The servo 5V line was connected to an external 5V power supply to ensure stable operation.

•The servo GND line was tied to the common ground shared with the ESP32-CAM.

•One leg of the pushbutton was connected to GPIO 2, configured as an input using the internal pull-up resistor.

•The other leg of the pushbutton was connected to GND, allowing GPIO 2 to read LOW when pressed.

•A shared GND was maintained between the ESP32 board, the servo motor, and the external 5V supply to keep a consistent reference.

For Tasks 2 and 3, the circuit was connected as follows:

•The ESP32-CAM 5V pin received power from a stable external 5V supply.

•The ESP32-CAM GND pin was connected to the common ground shared across all components.

•The Pan servo signal wire was connected to GPIO 14 for horizontal tracking movement.

•The Tilt servo signal wire was connected to GPIO 15 for vertical tracking movement.

•Both servo motors' 5V lines were connected to the external 5V supply.

•Both servo GND lines were connected to the common ground.

•A Dummy Servo 1 signal wire was connected to GPIO 12, required by the ESP32Servo library to avoid PWM timer conflicts with the camera.

•A Dummy Servo 2 signal wire was connected to GPIO 13 for the same purpose.

(These dummy servos do not need to be physically connected; only their signal pins are used.)

•During programming, the FTDI/USB-to-Serial RX pin was connected to U0T (GPIO 1) on the ESP32-CAM.

•The FTDI TX pin was connected to U0R (GPIO 3).

•The FTDI 5V pin was connected to the ESP32-CAM 5V pin only during code upload.

•The FTDI GND pin was connected to the ESP32-CAM GND.

•The IO0 pin was connected to GND to enter flashing mode, then disconnected after programming.

•A shared ground was maintained across the ESP32-CAM, servo motors, and the external power supply for stable operation.

4.0 METHODOLOGY

This section describes the experimental procedure for implementing a smart surveillance system using an ESP32-CAM module integrated with a servo motor for horizontal panning, web-based control, and brightness-based calm detection. The experiment involves both hardware and software configuration, demonstrating live video streaming, actuator control, and simple motion analysis using image brightness variations.

4.1 Objective

The objectives of this experiment were:

1. To understand the ESP32-CAM pin configuration and camera initialization procedure.
2. To control a servo motor for horizontal panning using PWM signals.
3. To stream live video over Wi-Fi through a built-in HTTP web server.
4. To implement a simple calm-detection algorithm based on image brightness stability.
5. To test manual and web-based control of the servo motor using a push button and a toggle button in the web interface.

4.1 Implementation and Testing

1. The camera was initialized with the appropriate AI Thinker pin mapping. A web server was started on port 80 to serve a real-time video stream and provide a simple user interface.

   Upon running the program, the ESP32-CAM printed its IP address through the Serial Monitor. Accessing this address through a web browser displayed:

- the live video stream from the camera,
- the servo ON/OFF status,

- calm detection status, and
- a control button to toggle servo movement.

The stream was verified for stability by adjusting parameters such as camera clock frequency, JPEG quality, and frame size.

2. A micro servo motor was attached to GPIO#14 and powered through an external 5V supply to ensure stable rotation.Two modes of servo control were tested:

   a. Automatic Panning

   When enabled, the servo continuously swept from 0° to 180° and back using PWM signals from the ESP32Servo library.
   Movement direction changed when endpoints were reached.

   b. Manual Button Toggle

   A push button connected to GPIO#12 allowed the user to toggle the servo movement.Debounce logic was added to prevent false triggering.Servo status changes were reflected instantly on the web interface.

3. To identify whether the environment was calm or moving, a simple brightness-based algorithm was implemented:

- A small portion of the JPEG frame buffer was sampled every 200 ms.
- The average brightness was computed and compared to the previous frame.
- If brightness changes remained below a threshold for several consecutive frames, the system flagged **CALM detected**.
- If brightness changed abruptly, the system marked **Movement detected**.

Whenever calm was detected, the servo motor automatically paused regardless of user control.Status updates were displayed on the web interface using periodic AJAX requests.

4.2 Control Algorithm

The ESP32-CAM was programmed using the Arduino framework to simultaneously handle video streaming, servo actuation, button input, and motion analysis.

The control algorithm operates as follows:

1. Initialize the ESP32-CAM using the OV2640 camera configuration and allocate separate LEDC timers for the camera clock and servo PWM.

2. Attach the servo motor to the specified GPIO pin and configure its PWM frequency (50 Hz).

3. Establish Wi-Fi connection using predefined SSID and password.

4. Start the HTTP server with endpoints for:

    - / → main webpage

    - /stream → MJPEG video stream

    - /toggle → enable/disable servo

    - /status → return JSON status (servo, calm detection)

5. Read the push button state using an internal pull-up resistor and toggle servo operation on button press.

6. For calm detection:
    a. Capture a camera frame every 200 ms.
    b. Sample pixels at intervals to estimate average brightness.
    c. Compute brightness differences between frames.
    d. Count stable frames to detect a calm environment.
    e. If calm detected, freeze servo movement.

7. For servo panning:
   a. If servo is enabled and calm is not detected, increment or decrement the angle.
   b. Reverse direction when endpoints (0° or 180°) are reached.
   c. Update servo position with a small delay for smooth movement.

8. Continuously update the web interface using JavaScript polling every second to reflect real-time system status.

4.4 Code Used

```
#include "esp_camera.h"
#include <WiFi.h>


#include "board_config.h"


const char *ssid = "Nip";
const char *password = "stonok123";

void startCameraServer();
void setupLedFlash();

void setup() {
  Serial.begin(115200);
  Serial.setDebugOutput(true);
  Serial.println();

  camera_config_t config;
  config.ledc_channel = LEDC_CHANNEL_0;
  config.ledc_timer = LEDC_TIMER_0;
  config.pin_d0 = Y2_GPIO_NUM;
  config.pin_d1 = Y3_GPIO_NUM;
  config.pin_d2 = Y4_GPIO_NUM;
  config.pin_d3 = Y5_GPIO_NUM;
  config.pin_d4 = Y6_GPIO_NUM;
  config.pin_d5 = Y7_GPIO_NUM;
  config.pin_d6 = Y8_GPIO_NUM;
  config.pin_d7 = Y9_GPIO_NUM;
  config.pin_xclk = XCLK_GPIO_NUM;
  config.pin_pclk = PCLK_GPIO_NUM;
  config.pin_vsync = VSYNC_GPIO_NUM;
  config.pin_href = HREF_GPIO_NUM;
  config.pin_sccb_sda = SIOD_GPIO_NUM;
  config.pin_sccb_scl = SIOC_GPIO_NUM;
  config.pin_pwdn = PWDN_GPIO_NUM;
  config.pin_reset = RESET_GPIO_NUM;
  config.xclk_freq_hz = 20000000;
  config.frame_size = FRAMESIZE_UXGA;
  config.pixel_format = PIXFORMAT_JPEG;
  config.grab_mode = CAMERA_GRAB_WHEN_EMPTY;
  config.fb_location = CAMERA_FB_IN_PSRAM;
```

```
  config.jpeg_quality = 12;
  config.fb_count = 1;


  if (config.pixel_format == PIXFORMAT_JPEG) {
    if (psramFound()) {
      config.jpeg_quality = 10;
      config.fb_count = 2;
      config.grab_mode = CAMERA_GRAB_LATEST;
    } else {
      config.frame_size = FRAMESIZE_SVGA;
      config.fb_location = CAMERA_FB_IN_DRAM;
    }
  } else {
    config.frame_size = FRAMESIZE_240X240;
#if CONFIG_IDF_TARGET_ESP32S3
    config.fb_count = 2;
#endif
  }

#if defined(CAMERA_MODEL_ESP_EYE)
  pinMode(13, INPUT_PULLUP);
  pinMode(14, INPUT_PULLUP);
#endif


  esp_err_t err = esp_camera_init(&config);
  if (err != ESP_OK) {
    Serial.printf("Camera init failed with error 0x%x", err);
    return;
  }

  sensor_t *s = esp_camera_sensor_get();
  if (s->id.PID == OV3660_PID) {
    s->set_vflip(s, 1);
    s->set_brightness(s, 1);
    s->set_saturation(s, -2);
  }
  if (config.pixel_format == PIXFORMAT_JPEG) {
    s->set_framesize(s, FRAMESIZE_QVGA);
  }
```

```
#if defined(CAMERA_MODEL_M5STACK_WIDE) ||
defined(CAMERA_MODEL_M5STACK_ESP32CAM)
  s->set_vflip(s, 1);
  s->set_hmirror(s, 1);
#endif

#if defined(CAMERA_MODEL_ESP32S3_EYE)
  s->set_vflip(s, 1);
#endif

#if defined(LED_GPIO_NUM)
  setupLedFlash();
#endif

  WiFi.begin(ssid, password);
  WiFi.setSleep(false);

  Serial.print("WiFi connecting");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("");
  Serial.println("WiFi connected");

  startCameraServer();

  Serial.print("Camera Ready! Use 'http://");
  Serial.print(WiFi.localIP());
  Serial.println("' to connect");
}

void loop() {
  delay(10000);
}


}
```

5.0 DISCUSSION

For Task 1, the push button successfully toggled the servo on and off, showing that the ESP32-CAM correctly processed digital input and responded with appropriate actuator movement. The servo reacted smoothly, indicating that the PWM setup and pull-up configuration were correctly implemented.

The web-based toggle control also worked reliably. The ESP32-CAM handled both video streaming and user commands without interruption, demonstrating that the HTTP server and JavaScript status updates were functioning as intended. The immediate change in servo behavior after each command showed good system responsiveness.

The calm-detection feature performed as expected. By comparing brightness between frames, the system was able to distinguish between movement and a stable scene. When the environment remained still, the servo automatically stopped, confirming that the detection logic was correctly integrated with the servo control. When movement was introduced, the servo resumed normal operation.

Overall, the system showed that the ESP32-CAM can stream video, detect basic motion, and control a servo at the same time. The results confirm that both the hardware setup and software logic worked together to produce a functional smart surveillance system.

6.0 CONCLUSION

This experiment successfully met its objectives by developing a functional smart surveillance prototype that integrates manual control, face detection, and dual-axis servo movement. The pushbutton interface provided simple manual panning control, while the face detection algorithm enabled automated pan-tilt adjustment based on the detected facial position.

The system performed reliably throughout all tasks, showing stable servo operation, accurate face detection, and effective environmental scanning when no target was present. These results demonstrate the capability of the ESP32-CAM as a cost-efficient solution for basic surveillance and object-tracking applications. Furthermore, the project highlights its value as an educational platform for mechatronics learning, particularly in areas involving sensor–actuator integration and embedded system development.

7.0 RECOMMENDATION

Some improvements can be made to enhance the system's performance. First, it is recommended to use a dedicated 5V power supply capable of providing at least 2A. Servo motors are highly sensitive to voltage instability, and insufficient current may cause voltage drops that affect both servo movement and camera functionality. A stable power source would ensure consistent operation, reduce interruptions during tracking, and improve overall system reliability.

Next, improving the smoothness of the tracking response is also suggested. During testing, sudden servo movements occurred when the detected face moved quickly or when the detection output fluctuated. Implementing a PID controller or applying a smoothing filter could minimize these abrupt changes, resulting in more precise and stable tracking. Additionally, although QVGA resolution enables faster processing, increasing the camera resolution to VGA or higher could improve detection accuracy especially in scenarios requiring finer detail.

Finally, the overall robustness of the system can be strengthened by adopting more advanced tracking techniques and improving mechanical stability. Incorporating object re-identification or color-tracking algorithms would help maintain target focus when multiple faces appear or during rapid movements. Mounting the system in a rigid and stable housing would also reduce vibration, which can affect both servo accuracy and camera detection. These enhancements would contribute to a more reliable, accurate, and efficient face-tracking system.

8.0 REFERENCE

[1] https://randomnerdtutorials.com/program-upload-code-esp32-cam/

How to Program / Upload Code to ESP32-CAM AI-Thinker (Arduino IDE)

[2] https://my.cytron.io/p-ch340-usb-to-Σl-serial-cable

CH340 USB to TTL Serial Cable

[3]https://arduino-er.blogspot.com/2020/09/program-esp32-cam-using-Ōdi-adapter.html

Program ESP32-CAM using FTDI adapter

[4] https://www.youtube.com/watch?v=D3MPBPGT3cw

Program ESP32-CAM using FTDI adapter

[5] https://core-electronics.com.au/guides/esp32-cam-set-up/

Use a ESP32-CAM Module to Stream HD Video Over Local Network

9.0 ACKNOWLEDGEMENT

We would like to sincerely thank the instructors, Assoc. Prof. Eur. Ing. Ir. Ts. Gs. Inv. Dr. Zulkifli Bin Zainal Abidin and the lab technicians, for all of their help,encouragement and support during this project. Their knowledge and perceptions have greatly influenced the course of this work. Additionally, we would like to express our gratitude to our peers for their support and cooperation, both of which were crucial to the accomplishment of this project.

10.0 STUDENT'S DECLARATION

We hereby declare that, except for the places where it is acknowledged, all of the work presented in this report is entirely ours. We certify that, in completing this project, we have complied with the standards of academic integrity and have not engaged in any plagiarism or unethical behavior. Every information and support source used in this work has been appropriately referenced and acknowledged.

Certificate of Originality and Authenticity

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgment, and that the original work contained herein has not been untaken or done by unspecified sources or persons. We hereby certify that this report has not been done by only one individual, and all of us have contributed to the report. The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have read and understand the content of the total report, and no further improvement on the reports is needed from any of the individual contributors to the report. We therefore agreed unanimously that this report shall be submitted for marking, and this final printed report has been verified by us.

Signature: ***ahmadfarhan***

Name: Ahmad Farhan bin Ahmad Nahrudi
Matric Card: 2317373

Read[/]
Understand[/]
Agree[/]

Signature: ***aimansafawi***

Name: Muhammad Aiman bin Safawi
Matric Card: 2318249

Read[/]
Understand[/]
Agree[/]

Signature: ***nafizzuhairee***

Name: Mohd Nafiz Zuhairee bin Mohd Neezar
Matric Card: 2317423

Read[/]
Understand[/]
Agree[/]